```c
#include <stdio.h>

int gcd(int a,int b){
    while(b!=0){
        int temp=b;
        b=a%b;
        a=temp;
    }
    return a;
}

int lcm(int a,int b){
    return (a/gcd(a,b))*b;
}

int find_lcm(int arr[],int n){
    int res=arr[0];
    for(int i=1;i<n;i++){
        res=lcm(res,arr[i]);
    }
    return res;
}

void sort(int n,int burst[],int period[]){
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(period[i]>period[j]){
                int temp;
                temp=period[i];
                period[i]=period[j];
                period[j]=temp;
                temp=burst[i];
                burst[i]=burst[j];
                burst[j]=temp;
            }
        }
    }
}

int main(){
    int n;
    printf("Enter the number of processes: ");
    scanf("%d",&n);
```

```c
int burst[n],period[n];

printf("Enter CPU burst times:\n");
for(int i=0;i<n;i++){
    scanf("%d",&burst[i]);
}

printf("Enter the time periods:\n");
for(int i=0;i<n;i++){
    scanf("%d",&period[i]);
}

sort(n,burst,period);

int hyper=find_lcm(period,n);
printf("\nLCM=%d\n",hyper);

printf("\nRate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for(int i=0;i<n;i++){
    printf("%d\t%d\t%d\n",i+1,burst[i],period[i]);
}

float util=0;
for(int i=0;i<n;i++){
    util+=(float)burst[i]/period[i];
}

float bound;
if(n==1) bound=1.0;
else if(n==2) bound=0.828427;
else if(n==3) bound=0.779763;
else bound=0.693;

printf("\n%.6f <= %.6f => %s\n",util,bound,(util<=bound)?"true":"false");
printf("Scheduling occurs for %d ms\n\n",hyper);

int remaining[n];
for(int i=0;i<n;i++) remaining[i]=0;

int last=-2;

for(int t=0;t<hyper;t++){
    for(int i=0;i<n;i++){
```

```c
            if(t%period[i]==0){
                remaining[i]=burst[i];
            }
        }

        int run=-1;
        for(int i=0;i<n;i++){
            if(remaining[i]>0){
                run=i;
                break;
            }
        }

        if(run!=last){
            if(run!=-1)
                printf("%dms onwards: Process %d running\n",t,run+1);
            else
                printf("%dms onwards: CPU is idle\n",t);
            last=run;
        }

        if(run!=-1) remaining[run]--;
    }

    return 0;
}
```

C:\Users\Admin\Desktop\1

```
Enter the number of processes: 2
Enter CPU burst times:
20 35
Enter the time periods:
50
100

LCM=100

Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 31.031 s
Press any key to continue.
```

1WA24CS232